CMCMIS_SIMPLIFIED

Phase 7 Slice 1 SEALED Documentation Admin Users + Employees + Session Revocation

DS-Style Senior Engineering Word Document

LOCKED SEAL

This document converts the pasted implementation transcript into a structured technical handover. It is written as a senior-engineering record: what was built, why it was built, how the database/backend/frontend logic fits together, and what evidence proves the phase is complete.

Item	Locked Insight
Phase	Phase 7 Slice 1
Scope	Admin Users, Employees master, token-version based session revocation, Super-Admin controls, frontend admin pages
Source artifact	Pasted text transcript uploaded in the current message
Documentation style	Step-by-step, insight-based, low-code, senior-engineering, DS locked format
Delivery posture	SEALED / LOCKED / handover-ready

0. Document Map

This document is intentionally structured like a production handover pack. It is not a raw transcript. It converts the implementation notes into an end-to-end engineering explanation.

Part	Section	Purpose
A	Context and phase identity	Defines what Phase 7 Slice 1 solved and why it exists.
B	Architecture and state logic	Explains token version, RBAC, session revocation, and admin mutation flow.
C	Database and migrations	Explains migration 110-113 and schema intent.
D	Backend modules	Explains adminUsers, employees, auth middleware, repositories, services, controllers.
E	Frontend modules	Explains API wrappers, hooks, schemas, pages, route wiring.
F	Smoke testing and acceptance	Documents A1-A15 proof and verification logic.
G	Senior engineering notes	Risk register, maintenance rules, next-phase readiness.
H	Appendices	Endpoints, file inventory, matrices, transcript-derived decisions.

1. Phase Identity and Locked Objective

Phase 7 Slice 1 is the administrative-control layer of CMCMIS_SIMPLIFIED. It gives the system a controlled Super-Admin cockpit for users, employees, role transitions, activation/deactivation, account creation, forced logout, and session revocation.

Definition Block - What this phase means

Phase 7 Slice 1 is not a generic admin screen. It is the security and personnel-control foundation that allows future modules to trust who a user is, which role they hold, whether their session is still valid, and whether their employee record is active.

Item	Locked Insight
Primary user	SUPER_ADMIN
Main screens	/admin/users, /admin/employees, /admin/employees/new, /admin/employees/:id/edit
Main database area	users, user_roles, roles, permissions, role_permissions, cmms_emp_mst, user_role_history, audit_log
Core security mechanism	JWT carries token_version; authenticate compares token claim against DB/cache current version
Core operational value	One place to control user role, account status, employee master data, and forced session invalidation

Locked Principle

Every administrative mutation must be auditable, permission-gated, transaction-safe, and capable of revoking affected sessions when identity or access changes.

2. Why Phase 7 Slice 1 Was Necessary

Before this phase, login and role loading existed, but operational administration was incomplete. A production system needs a controlled way to manage users and employees without editing database rows manually.

- Super Admins require a real user list with filtering, status indicators, and per-row actions.
- Role changes must not silently leave old JWTs valid; the old session must be invalidated after the permission surface changes.
- Employee master records must support create, update, soft-delete, and account creation through a strict workflow.
- The system needs history and audit logs so administrative actions are reviewable later.
- Frontend navigation must reveal Super-Admin-only screens only to eligible users.

Senior Engineering Insight

The important idea is not merely “build admin pages.” The real goal is to create identity governance. Once identity governance is stable, every future module can safely rely on role-based decisions.

3. Sealed Inputs and Grounding Context

The pasted transcript shows the implementation was grounded in the previously sealed Phase 6 and Phase 7 context, then expanded into a complete admin module.

Item	Locked Insight
Previously sealed state	Phase 6 Slice 1 and Phase 7 schema context were read before implementation.
Project root checked	Claude Code Documents, DB, Documents, INSIGHTS, SOFTWARE CODE, docx word files.
Backend modules found	adminUsers, auth, employees, equipment, jobCards, jobRequests, lookups, users.
Discovery docs found	SCHEMA_PHASE6.md, SCHEMA_PHASE7.md, ROUTES_PHASE6.md, 0001_phase6_introspect.sql.
Final-desc located	FINAL-DESC-CMCMIS.pdf and phase sealed PDFs in project folders.

Locked Context Rule

This document treats the pasted transcript as the source of truth for Phase 7 Slice 1 completion. It does not redesign the system; it documents what was done and why it is technically correct.

4. System Flow at a Glance

The central flow is: Super Admin opens a protected page, performs a mutation, backend authenticates and authorizes, service validates invariants, database transaction writes changes/history/audit, then token cache invalidation forces stale sessions out.

```
SUPER ADMIN (browser)
   |
   v
/admin/users or /admin/employees
   |
   v
API call -> authenticate -> token_version check -> authorize -> validate
   |
   v
controller -> service -> state-machine guard -> repository transaction
   |
   +-- update core table
   +-- write history row
   +-- write audit_log row
   +-- bump token_version when access/session impact exists
   |
   v
commit -> invalidate tokenVersionCache -> return normalized response
```

Architecture Decision

The phase uses a layered architecture: route middleware handles access gates, controllers remain thin, services own business invariants and transactions, repositories own SQL, and frontend hooks own caching/refetch behavior.

5. Security Architecture - token_version Session Revocation

The strongest technical addition in this phase is token_version based revocation. JWT alone is stateless, so old tokens can otherwise remain valid even after role/status changes. token_version fixes that.

Item	Locked Insight
JWT claim	tv - token_version at the time of token issue
Database source	users.token_version
Cache utility	tokenVersionCache.js with 5000-entry LRU and 30-second TTL
Middleware patch	authenticate.js reads claim tv, compares against current tv, rejects stale tokens
Revocation response	401 with reason/session revoked semantics; frontend redirects to login

Definition - Session Revocation

A session is considered revoked when the token_version inside its JWT is lower than the current token_version stored for that user. The token may still be cryptographically valid, but it is no longer authorization-valid.

```
Pseudo-flow:
1. User logs in -> JWT issued with tv = users.token_version.
2. Super Admin changes role/deactivates/force-logs out user.
3. Backend increments users.token_version.
4. tokenVersionCache invalidates that user.
5. User sends old JWT on next request.
6. authenticate detects token.tv < current_tv.
7. API returns 401 SESSION_REVOKED.
8. FE clears auth state and redirects to login.
```

6. tokenVersionCache Design

The transcript shows a cache-first strategy for token version checks. This avoids turning every authenticated request into a direct database read while still keeping revocation latency bounded.

Item	Locked Insight
Cache size	5000 entries
TTL	30 seconds
Miss behavior	Read token_version from users table and write cache
Invalidation behavior	Invalidate after administrative mutations that bump token_version
Upgrade behavior	Older tokens without tv default to 1, giving a graceful migration path

Performance Insight

Without caching, every protected endpoint would query the users table on every request. With a short TTL LRU, normal traffic remains fast while administrative changes can still invalidate a user immediately through explicit cache deletion.

7. Authentication Middleware Patch

authenticate.js was patched to enforce token version comparison after JWT verification. This is the correct layer because every protected route already flows through authenticate before permission checks.

1. Parse and verify access token.
2. Extract user_id, employee_id, role, permissions, and tv claim.
3. Resolve current token_version through tokenVersionCache or database.
4. Compare claim tv with current tv.
5. Reject stale sessions before route authorization runs.
6. Continue request only if token is both cryptographically valid and version-current.

Security Rule

Authorization must never be checked using a stale identity surface. The token_version check ensures that a role-changed or deactivated user cannot continue using old permissions.

8. Frontend SESSION_REVOKED Handling

The frontend api-client was patched to detect SESSION_REVOKED and cleanly recover. This prevents confusing partial failures and gives the user a clear re-login path.

Item	Locked Insight
Trigger	Backend returns 401 with session revoked reason
Frontend action	Clear auth tokens/context
Navigation	Redirect to /login?reason=session_revoked
User experience	Old tab is bounced to login after role/status mutation
Engineering benefit	No stale UI state, no hidden permission mismatch

UX Insight

Security failures should be understandable. Redirecting with a reason allows the login page to explain that the session expired because access was changed, not because of a random network error.

9. Database Design Philosophy for This Phase

The phase follows the established CMCMIS doctrine: do not damage legacy tables, add only required columns/tables, use stable legacy keys, write audit/history records for high-impact changes.

- Use existing users and user_roles for account and role association.
- Use existing cmms_emp_mst as the employee master instead of inventing a new employees table.
- Use user_role_history for traceability of role/status changes.
- Use audit_log for generic operational action records.
- Add only the metadata needed for token revocation and soft-delete tracking.

Database Logic Development

The database design is not table-first; it is workflow-first. First identify the actor, action, invariants, affected tables, and rollback boundary. Then decide which tables need new fields and which writes must happen in the same transaction.

10. Migration 110 - Users and Employee Metadata Columns

Migration 110 introduced the core columns needed for Phase 7 runtime behavior. The most important additions are token_version and employee deactivation timestamp metadata.

Item	Locked Insight
users.token_version	Integer counter used to revoke old JWTs after role/status changes.
users.force_password_change	Supports account bootstrap and future password governance.
users.is_locked / failed_login_count	Supports account lock logic and future security hardening.
users.deactivation_reason	Captures why a user was deactivated.
cmms_emp_mst.EMM_DEACTIVATED_AT	Timestamp when employee soft-delete flag was flipped.

Migration Rule

Every new field was additive. Existing legacy records remain compatible. This is essential because CMCMIS contains real historical data and cannot tolerate destructive schema edits.

11. Migration 111 - Indexing Strategy

Migration 111 added indexes for the admin list and employee list workloads. Search, filtering, role lookup, and active/inactive filtering must remain fast as user and employee records grow.

Index Target	Why It Exists	Expected Query Shape
users.token_version / status fields	Fast revocation and status filtering	WHERE user_id = ? or is_active = ?
role joins	Admin user listing includes role names/codes	JOIN user_roles -> roles
employee active/department	Employee list filters by active state and division	WHERE EMM_INACTIVE = ? AND EMM_DEPT = ?
history tables	History endpoint retrieves transitions quickly	WHERE user_id = ? ORDER BY changed_at DESC

Performance Principle

Indexes were created according to real screen behavior: list pages, filters, detail pages, and history views. Good enterprise UI speed begins in SQL shape, not in React rendering.

12. Migration 112 - user_role_history

The user_role_history table records administrative identity transitions. This separates identity-change history from generic audit_log while allowing a clean UI history endpoint.

Item	Locked Insight
Purpose	Record role changes, activation/deactivation, and force-logout-style events.
Actor	Super Admin employee ID who performed the operation.
Target	Affected user/employee.
Reason	Optional or required reason depending on action.
Read model	Admin user detail/history endpoint displays the timeline.

Traceability Rule

If an action changes access, identity, employment status, or active session validity, it must leave a history row plus an audit row.

13. Migration 113 - Permissions

Migration 113 seeded the extra Super-Admin permissions needed for user and employee administration. The transcript notes that permission count moved from the older bootstrap expectation to 43, which is expected after this phase.

Permission Area	Representative Permission	Role Granted
User list/detail	user:read-list	SUPER_ADMIN
Role assignment	user:role-assign	SUPER_ADMIN
Activate/deactivate	user:activate / user:deactivate	SUPER_ADMIN
Force logout	user:force-logout	SUPER_ADMIN
Employee CRUD	employee:create / employee:update / employee:delete	SUPER_ADMIN
Account creation	employee:create-account	SUPER_ADMIN

Bootstrap Verification Note

The legacy verification script still expected exactly 40 permissions and 2 users. After Phase 7, the database intentionally contains 43 permissions and around 60 users. The verification mismatch is a script expectation drift, not a failed migration.

14. Backend Module Pattern Used in Phase 7 Slice 1

The phase follows the mature project backend pattern: validators, state machine, repository, service, controller, routes. This keeps business logic clear and testable.

```
adminUsers/
  adminUsers.validators.js
  adminUsers.stateMachine.js
  adminUsers.repo.js
  adminUsers.service.js
  adminUsers.controller.js
  adminUsers.routes.js

employees/
  employees.validators.js
  employees.repo.js
  employees.service.js
  employees.controller.js
  employees.routes.js
```

Engineering Rule

Controllers should not contain business policy. Services should not contain raw SQL. Repositories should not decide business invariants. This separation is what makes the phase maintainable.

15. Admin Users Module - Purpose and Responsibilities

The adminUsers module is the control plane for existing application users. It allows Super Admins to list users, view details, view history, change role, activate/deactivate, and force logout.

Endpoint Intent	What It Does	Critical Guard
List users	Paginated/filterable list with role/status info	user:read-list
User detail	One user profile and account state	user:read-list
User history	Role/status transition timeline	user:read-list
Change role	Updates user_roles and bumps token_version	No self-role change; Super Admin invariant
Activate/deactivate	Toggles user account status and bumps token_version	No self-deactivate; last-SA guard
Force logout	Bumps token_version without changing role	Super Admin only

Critical Insight

The admin user module is not only CRUD. It is access mutation. Every mutating operation can change what the target user can do, so session invalidation is mandatory.

16. Admin Users State Machine and Invariants

The state machine holds the pure rules for role/status mutations. By making the invariants explicit, the backend avoids subtle self-lockout and privilege-loss bugs.

Invariant	Meaning	Why It Matters
I-1 Role transition validity	Only allowed role changes pass	Prevents corrupt or undefined role state
I-3 Self role-change forbidden	Super Admin cannot change their own role	Prevents accidental privilege loss
I-4 Self deactivate forbidden	Super Admin cannot deactivate their own account	Prevents immediate lockout
Last active SA guard	System must retain an active Super Admin	Guarantees recoverability
Force logout is access-impacting	Bumps token_version even without role/status change	Ends stale sessions deterministically

Senior Developer Reasoning

A Super Admin module must be designed around failure scenarios first: self-demotion, self-deactivation, last-admin deletion, stale token continuation, and untracked changes.

17. Admin Users Transaction Pattern

The role-change flow illustrates the transaction model used throughout the phase. All related writes succeed together or fail together.

```
Role change transaction:
1. guardRoleChange(target, newRole, actor)
2. BEGIN
3. UPDATE user_roles
4. UPDATE users SET token_version = token_version + 1
5. INSERT user_role_history
6. INSERT audit_log
7. COMMIT
8. tokenVersionCache.invalidate(targetUserId)
```

Transaction Rule

Never update the role without bumping token_version. Never bump token_version without history/audit when the operation is administrative. These writes belong together.

18. Employees Module - Purpose and Responsibilities

The employees module manages cmms_emp_mst as the authoritative employee master. It supports list, detail, create, update, soft-delete, and account creation.

Function	Database Effect	Operational Meaning
List employees	Reads cmms_emp_mst joined with account/role data	Admin overview of people and accounts
Create employee	Inserts cmms_emp_mst row	Adds a person to the organization master
Update employee	Updates allowed EMM_* fields	Maintains designation, division, contact, profile data
Soft-delete employee	Sets EMM_INACTIVE and EMM_DEACTIVATED_AT	Keeps historical data but hides inactive employees
Create account	Creates users + user_roles for an employee	Converts employee record into login-capable account

Database Insight

The system wisely reuses cmms_emp_mst instead of creating a duplicate employee table. This protects historical compatibility and avoids two sources of truth for employees.

19. Employee Soft-Delete Logic

Employee deletion is implemented as soft-delete. This is essential in an engineering maintenance system because employee IDs may exist in historical job requests, job cards, audit logs, and approval records.

- Soft-delete flips EMM_INACTIVE rather than removing the employee row.
- EMM_DEACTIVATED_AT records when the employee was deactivated.
- Soft-delete is blocked when an active user account exists, preventing orphan login/account state.
- Historical reports can still resolve employee references.

Risk Avoided

Hard-deleting employees would break historical joins and audit evidence. Soft-delete preserves the institutional memory of the system.

20. Employee Account Creation Flow

The employee account creation flow creates login capability for a fresh employee record. It returns a one-time initial password and avoids duplicate accounts.

```
Employee -> Create Account:
1. Super Admin opens employee row.
2. Backend checks employee exists and is active.
3. Backend checks employee does not already have a user account.
4. Backend generates one-time 12-character initial password.
5. Backend creates users row and user_roles row.
6. UI shows the initial password once in a warning callout.
```

Security Insight

Showing the generated password once is intentional. It reduces accidental leakage and pushes the admin to copy/share it through an approved internal process.

21. Auth Service and Users Repository Patch

The source transcript shows auth was patched so JWTs carry token_version. This connects login-time identity creation to request-time revocation checking.

Item	Locked Insight
Login service	Reads token_version during login and embeds it in JWT claim tv.
Users repository	Provides token_version lookup and mutation helpers.
Admin services	Bump token_version after role/status/force-logout changes.
Middleware	Reads token claim and checks current version.

End-to-End Identity Chain

Login -> JWT tv -> authenticate -> DB/cache tv -> admin mutation -> tv bump -> cache invalidation -> old session rejected. Every link is required.

22. Server Route Wiring

server.js was patched to mount the new admin modules. This exposes the backend implementation through stable API paths.

Route Prefix	Module	Purpose
/api/v1/admin/users	adminUsers.routes.js	User account administration
/api/v1/admin/employees	employees.routes.js	Employee master administration

Routing Rule

Only mount a module after auth, authorization, validation, service, and repository layers are ready. This keeps half-built modules from leaking into runtime.

23. Frontend Architecture Overview

The frontend work mirrors the backend: API wrappers, zod schemas, hooks with caching, permission-driven navigation, and page components with inline modals.

Frontend Layer	Files	Responsibility
API wrappers	adminUsers.js, employees.js	Centralized HTTP calls
Schemas	adminUserSchemas.js, employeeSchemas.js	Frontend validation mirrors backend zod
Hooks	useAdminUserList, useEmployeeList	30-second TTL list caching
api-client	interceptor patch	Handle SESSION_REVOKED globally
Permissions/nav	sidebar permissions patch	Show Super-Admin-only entries
Pages	UserList, EmployeeList, EmployeeForm	Admin UI workflow

Frontend Design Principle

Frontend validation is helpful, but backend validation is authoritative. The FE zod schemas reduce friction; BE zod schemas protect data.

24. UserList Page

The UserList page gives the Super Admin a table of all users with role pills, status pills, last-login data, and actions for role change, activation/deactivation, and force logout.

- Role pills make identity classification visually fast.
- Status pills show active/inactive/locked-like account conditions.
- Inline modal actions reduce navigation overhead.
- List data is cached for 30 seconds to reduce repeated API load.
- Mutations invalidate relevant cached views and update the display.

UX Insight

An admin table should not just show rows; it should show operational state and provide safe, contextual actions without making the Super Admin jump across pages.

25. EmployeeList Page

EmployeeList is the master-data cockpit for people records. It supports filtering, New Employee flow, edit flow, soft-delete, and create-account action.

UI Action	Backend Operation	Important Rule
+ New Employee	POST employee	Employee ID pattern must match locked format
Edit	PATCH employee	Only allowed fields are mutable
Soft Delete	PATCH/DELETE style soft-delete	Blocked if active account exists
Create Account	POST account creation	Duplicate account blocked

Operational Note

Employee records and user accounts are related but not identical. A person can exist as an employee before they have a login account.

26. EmployeeForm Page

EmployeeForm handles both create and edit modes. It uses a shared schema and presents employee master fields in a consistent government-grade administrative layout.

Item	Locked Insight
Create mode	Builds a new cmms_emp_mst record.
Edit mode	Loads existing employee detail and patches changed fields.
Validation	Employee ID pattern, full name, designation, division, email/mobile/date constraints.
Navigation	Wired under /admin/employees/new and /admin/employees/:id/edit.

Form Design Rule

A master-data form should be boring in the best way: predictable, validated, minimal surprise, and consistent with backend constraints.

27. Frontend Route Table

App.jsx was patched to include Super-Admin-only routes. This makes the admin module reachable in the UI while preserving permission boundaries.

Route	Page	Visibility
/admin/users	UserList	Super Admin only
/admin/employees	EmployeeList	Super Admin only
/admin/employees/new	EmployeeForm create mode	Super Admin only
/admin/employees/:id/edit	EmployeeForm edit mode	Super Admin only

Navigation Insight

Route protection must exist at two levels: frontend for UX and backend for security. Hiding a link is not security; backend authorization is.

28. Frontend Cache and Refetch Strategy

The source transcript notes 30-second TTL hooks for admin and employee lists. This is a pragmatic balance between responsive UI and backend load.

Item	Locked Insight
useAdminUserList	Caches admin user list results and refetches after mutation or TTL expiry.
useEmployeeList	Caches employee master list results and supports filter changes.
Mutation behavior	Invalidate or refetch affected lists after role/status/employee changes.
Session revoked behavior	Global interceptor clears session and redirects to login.

Performance/UX Principle

Admin pages should feel live without becoming chatty. A 30-second TTL is enough for administrative screens where immediate millisecond freshness is less important than stability.

29. Smoke Verification Strategy

The phase was not considered complete until the A1-A15 smoke matrix ran. These tests prove authentication, authorization, invariants, token versioning, history, and frontend build readiness.

Test Group	What It Proves
A1-A3	Authentication and self-protection gates work.
A4-A8	Super Admin invariants and deactivation protections exist.
A9-A13	Deactivate, duplicate employee, duplicate account, and fresh account flows work.
A14	tokenVersionCache warm reads are fast.
A15	History endpoint records administrative transitions.

Testing Principle

Smoke tests are not only happy-path checks. For admin/security phases, forbidden paths are more important than successful paths.

30. A1-A15 Smoke Matrix

The pasted transcript included a final verification table. The following converts it into the documentation acceptance matrix.

ID	Criterion	Expected/Verified Result
A1	GET /admin/users without auth	401; no token equals no access
A2	Super Admin sees all users	200; total_items visible
A3	Super Admin changes own role	409 SELF_MODIFICATION_FORBIDDEN
A4	Last-SA invariant infrastructure	Two active SAs visible as prerequisite
A5	NORMAL -> LAB_ENGINEER	200 role transition returned
A6	tokenVersion bumped and invalidated	tv increments after role/deactivation mutations
A7	Super Admin deactivates self	409 SELF_DEACTIVATE_FORBIDDEN
A8	Last-active-SA deactivate	Guard logic wired
A9	Deactivate user with reason	200; is_active=false, deactivated_at and reason present
A10	Soft-delete active user blocked	GuardEmployeeSoftDelete invariant
A11	Duplicate employee ID	409 CONFLICT on employee_id
A12	Create account when account already exists	409 CONFLICT HAS_ACCOUNT
A13	Create account fresh employee	Initial password returned once
A14	LRU cache warm reads	5 reads <= 16ms total
A15	History endpoint	DEACTIVATE and CHANGE_ROLE rows visible

31. Build Verification

The frontend production build completed cleanly, transforming 1690 modules with zero errors. This matters because the admin pages were not just backend-ready; they were integrated into the application shell.

Item	Locked Insight
Frontend build	1690 modules transformed
Status	Zero errors
New admin pages	UserList, EmployeeList, EmployeeForm
Route integration	App.jsx patched with 4 routes
Sidebar/nav	Super-Admin-only entries patched

Delivery Proof

A backend-only phase would not be sealed. This phase reached full-stack readiness: DB migrations, backend routes, frontend views, route wiring, and build verification.

32. Senior Logic Development - How to Think About the Backend

The backend was developed by reasoning from business invariants to database operations. This is the correct approach for admin/security workflows.

1. **1.** Identify the actor: Super Admin or not.
2. **2.** Identify the target: user account or employee record.
3. **3.** Identify the action: role change, deactivate, activate, force logout, create employee, update employee, soft-delete, create account.
4. **4.** Identify hard invariants: no self-demotion, no self-deactivation, no last-SA lockout, no duplicate account, no hard-delete employee.
5. **5.** Identify the transaction boundary: every write that must remain consistent goes inside one transaction.
6. **6.** Identify session impact: if permissions or active status change, bump token_version.
7. **7.** Identify audit impact: write history and audit rows for administrative operations.
8. **8.** Invalidate cache only after commit.

Senior Engineering Rule

Never start with controller code. Start with invariants, then transaction shape, then SQL, then service, then controller, then frontend.

33. Senior Logic Development - How to Think About the Database

Database changes in this phase are minimal but high leverage. Each new column/table exists because a workflow needed durable state.

Workflow Need	Database Decision	Why
Revoke sessions after access mutation	users.token_version	Reliable, cheap, deterministic revocation
Explain why deactivation happened	deactivation_reason and history/audit rows	Audit and accountability
Know when employee was soft-deleted	EMM_DEACTIVATED_AT	Historical timeline
Review role/status transitions	user_role_history	Dedicated identity timeline
Keep historical employee references	Soft-delete instead of hard-delete	Preserves legacy reports and audit joins

Database Principle

If a value must influence security, workflow, audit, or historical reporting, it deserves explicit durable storage. If it is only UI decoration, it should not bloat the schema.

34. Senior Logic Development - How to Think About Frontend

The frontend was built around safe operations and clear user feedback. This is especially important in Super Admin screens because every action can affect access.

- Use role/status pills to reduce cognitive load.
- Use modals for dangerous actions so the admin pauses and confirms.
- Show generated passwords once and visually emphasize the responsibility.
- Handle SESSION_REVOKED globally so stale sessions never create confusing local failures.
- Keep frontend routes protected but never rely on route protection alone.

Frontend Principle

Admin UX must be efficient but never careless. Dangerous operations need confirmation, clear labels, and immediate feedback.

35. Risk Register

This section documents the main risks addressed by the phase and the mitigation implemented.

Risk	Impact If Unfixed	Mitigation in Phase 7 Slice 1
Old JWT remains valid after role change	User keeps old powers	token_version claim + DB comparison + cache invalidation
Super Admin locks themselves out	System admin deadlock	Self role-change/deactivate guards
Last Super Admin deactivated	No recovery admin left	Last-active-SA invariant
Employee hard-delete breaks history	Reports/audit rows lose actor meaning	Soft-delete via EMM_INACTIVE and timestamp
Duplicate account for employee	Conflicting login identity	Create-account guard
Silent admin changes	No auditability	user_role_history + audit_log rows

36. Operational Handover Checklist

This checklist can be used by a senior developer or reviewer before accepting the phase as sealed.

Checklist Item	Expected State
Migrations 110-113 applied	Yes
permissions count drift understood	Yes - expected after Phase 7
tokenVersionCache in place	Yes, LRU + TTL
authenticate.js tv check active	Yes
adminUsers routes mounted	Yes
employees routes mounted	Yes
Super Admin sidebar entries visible only to SA	Yes
SESSION_REVOKED interceptor active	Yes
A1-A15 smoke green	Yes
Frontend build clean	Yes

Acceptance Statement

Phase 7 Slice 1 is accepted when security invariants, database writes, audit history, frontend pages, route gates, cache invalidation, and smoke tests all align.

37. What This Phase Enables Next

With Admin Users and Employees sealed, later phases can depend on stronger identity and personnel management.

- Lab Engineer assignments become easier because employees/accounts are manageable from UI.
- Role-based dashboards can trust current roles instead of stale tokens.
- Future audit/activity modules can read meaningful admin history.
- SSO integration can reuse the user/employee separation and token_version revocation model.
- Reports can join employee names without worrying about hard-deleted people.

Next-Phase Readiness

This phase unlocks reliable RBAC operations for all upcoming modules: dashboard, inquiry, job request conversion, job card execution, and reports.

Appendix A - Deliverables Snapshot

The transcript included a deliverables snapshot. This table preserves it in documentation form.

Area	Files / Unit	Status
DB	SCHEMA_PHASE7.md + migrations 110-113	Applied + verified
BE utils	tokenVersionCache.js	LRU 5000 / 30s TTL
BE middleware	authenticate.js	token_version check + SESSION_REVOKED
BE adminUsers	6 files	Live, 7 endpoints
BE employees	5 files	Live, 6 endpoints
BE server	server.js	2 new app.use lines
BE auth	users.repo + auth.service	JWT carries tv claim
FE api	adminUsers.js, employees.js	Wrappers complete
FE schemas	adminUserSchemas.js, employeeSchemas.js	zod mirror
FE hooks	useAdminUserList, useEmployeeList	30s TTL cache
FE api-client	interceptor patch	SESSION_REVOKED handler
FE permissions	navigation patch	2 SA-only items
FE pages	UserList, EmployeeList, EmployeeForm	Inline modals
FE App.jsx	4 admin routes	Wired
FE build	1690 modules	Zero errors

Appendix B - Endpoint Inventory

This is a compact endpoint view for reviewers and future backend work.

Module	Endpoint	Purpose
Admin Users	GET /admin/users	List users
Admin Users	GET /admin/users/:id	User detail
Admin Users	GET /admin/users/:id/history	User history
Admin Users	PATCH /admin/users/:id/role	Change role
Admin Users	PATCH /admin/users/:id/activate	Activate user
Admin Users	PATCH /admin/users/:id/deactivate	Deactivate user
Admin Users	POST /admin/users/:id/force-logout	Bump token_version
Employees	GET /admin/employees	List employees
Employees	GET /admin/employees/:id	Employee detail
Employees	POST /admin/employees	Create employee
Employees	PATCH /admin/employees/:id	Update employee
Employees	DELETE/PATCH soft-delete	Soft-delete employee
Employees	POST /admin/employees/:id/account	Create user account

Appendix C - Role and Permission Matrix

Phase 7 Slice 1 admin operations are Super-Admin-only.

Operation	SUPER_ADMIN	LAB_IN_CHARGE	LAB_ENGINEER	NORMAL_USER	VIEW_ONLY
View admin users	Allowed	Denied	Denied	Denied	Denied
Change user role	Allowed	Denied	Denied	Denied	Denied
Activate/deactivate user	Allowed	Denied	Denied	Denied	Denied
Force logout user	Allowed	Denied	Denied	Denied	Denied
Create/update employee	Allowed	Denied	Denied	Denied	Denied
Soft-delete employee	Allowed	Denied	Denied	Denied	Denied
Create employee account	Allowed	Denied	Denied	Denied	Denied

Permission Reminder

This matrix documents UI intent. Backend authorize middleware remains the source of enforcement.

Appendix D - Important User-Facing Browser Behavior

The transcript describes what should be visible in the browser after completion.

- Open /admin/users: Super-Admin-only sidebar entry appears.
- Table shows users with role pills, status pills, last-login, and row actions.
- Role icon opens modal to select new role and reason.
- Backend bumps token_version and writes audit/history after role change.
- Old user tab is redirected to login with session_revoked reason.
- Open /admin/employees: CRUD master appears with New Employee and edit flows.
- Create Account icon generates a one-time password shown in a yellow callout.

Appendix E - Raw Transcript Digest

The following digest preserves the most important source-derived final summary without dumping the entire code-heavy transcript.

```
PHASE 7 SLICE 1 - END-TO-END SUMMARY
Architecture flow

+-------------------------------------------------------------------------+
|  SUPER ADMIN  (browser)                                                 |
|       |                                                                 |
|       ▼                                                                 |
|  /admin/users  --->  PATCH /admin/users/:id/role  --->  authenticate     |
|                                                        (verify tv vs cache)  |
|                                                             |           |
|                                                             ▼           |
|                                                        authorize        |
|                                                        (user:role-assign)    |
|                                                             |           |
|                                                             ▼           |
|                                                        validate (zod)   |
|                                                             |           |
|                                                             ▼           |
|  +---------------  controller  --->  service  ----------------------------+|
|  |  1. stateMachine.guardRoleChange()   -- I-1, I-3 - pure                ||
|  |  2. BEGIN                                                              ||
|  |  3. repo.changeUserRole         (UPDATE user_roles)                    ||
|  |  4. repo.bumpTokenVersion       (UPDATE users SET tv = tv + 1)         ||
|  |  5. repo.appendHistory          (INSERT user_role_history)             ||
|  |  6. repo.writeAudit             (INSERT audit_log)                     ||
|  |  7. COMMIT                                                              ||
|  |  8. tokenVersionCache.invalidate(id)                                   ||
|  +------------------------------------------------------------------------+|
|       |                                                                 |
|       ▼                                                                 |
|  Bob's next request (his OLD JWT) -> authenticate sees claim tv < db tv     |
|                                -> 401 { reason: 'SESSION_REVOKED' }       |
|  FE api-client intercepts SESSION_REVOKED -> wipes tokens -> /login      |
+-------------------------------------------------------------------------+
Verified live (A1-A15)
#       Criterion       HTTP    Verification
A1      GET /admin/users without auth    401     no token = no access
A2      SA sees all 60 users     200     total_items=60, items returned
A3      SA changes own role (uid=1)      409 SELF_MODIFICATION_FORBIDDEN  invariant I-3
A4      last-SA invariant infrastructure 2 active SAs visible     countActiveSuperAdmins=2 (test condition
prereq)
A5      NORMAL -> LAB_ENGINEER on user 57        200     {"id":57,"role":"LAB_ENGINEER"}
A6      tokenVersion bumped & invalidated        tv 1->2->3      after role change + deactivate
A7      SA deactivates self      409 SELF_DEACTIVATE_FORBIDDEN    invariant I-4
A8      last-active-SA deactivate        wired in guardDeactivate (same logic as A4)
A9      deactivate user 57 + reason      200     is_active=false, tv=3, deactivated_at=…, reason=…
A10     soft-delete with active user blocked     wired in guardEmployeeSoftDelete invariant I-5
A11     duplicate emp_id on POST employees       409 CONFLICT    field: employee_id
A12     create-account when already has one      409 CONFLICT    details.code: HAS_ACCOUNT
A13     create-account fresh employee    wired in employees.service.createAccount returns initial_password once
A14     LRU cache 30s TTL warm   5 reads ≤ 16ms total     warm cache hit
A15     History endpoint records transitions     2 rows visible   DEACTIVATE + CHANGE_ROLE rows
Deliverables snapshot
Area    Files   Status
DB      SCHEMA_PHASE7.md + 4 migrations (110-113)        applied + verified
BE utilstokenVersionCache.js     LRU 5000/30s
BE middleware    authenticate.js patched  tv check + SESSION_REVOKED
BE adminUsers    6 files (validators / stateMachine / repo / service / controller / routes)        live, 7
endpoints
BE employees     5 files (validators / repo / service / controller / routes)      live, 6 endpoints
BE server        server.js patched       2 new app.use lines
BE auth users.repo + auth.service patched        JWT carries tv claim
FE api  adminUsers.js, employees.js      wrappers complete
FE schemas       adminUserSchemas.js, employeeSchemas.js  zod mirror
FE hooksuseAdminUserList, useEmployeeList        30s TTL cache
FE api-client    patched SESSION_REVOKED handler
FE permissions   nav patched      2 SA-only items
```

```
FE pagesUserList, EmployeeList, EmployeeForm     with inline modals
FE App.jsx      4 new routes wired       /admin/users, /admin/employees, /new, /:id/edit
FE build1690 modules transformed zero errors
What you'll see in the browser
Open /admin/users - Super-Admin-only sidebar entry visible. Table of all 60 users with role pills, status pills,
last-login, and per-row act
Click role icon -> modal opens -> pick new role + optional reason -> save. Backend bumps token_version, writes
audit + history.
Within 30 seconds, that user's old tab refreshes -> gets 401 SESSION_REVOKED -> bounced to
/login?reason=session_revoked.
Open /admin/employees - full CRUD master with + New Employee, edit, soft-delete, and a "create account" UserPlus
icon that opens a modal gen
Backend is currently down (pkill at end of smoke). Boot with cd "SOFTWARE CODE/BE" && node src/server.js, FE with
cd "SOFTWARE CODE/FE" && n
```

Appendix F - Documentation Seal

This final page closes the Phase 7 Slice 1 documentation pack.

SEALED STATUS

Phase 7 Slice 1 is documented as an end-to-end completed slice: database migrations, backend access-control modules, employee master operations, session revocation, frontend routes/pages, and A1-A15 smoke verification. Future phases should treat this as a stable identity-governance foundation unless explicitly reopened.

Item	Locked Insight
Document generated	2026-05-19 13:02 UTC
Source	Uploaded pasted text transcript
Output	phase7sliceOneSEALED_AdminUsersEmployees_SessionRevocation.docx
Seal	PHASE 7 SLICE 1 - ADMIN USERS + EMPLOYEES - LOCKED